Source: README.md | Export: 11 November 2026

> **Current consolidated documentation:** Parinaya Sarees documentation index. The index separates the inspected local implementation, existing delivery changes, historical milestone notes and items pending Production/account confirmation. The earlier README below is retained as historical setup context; do not use its obsolete feature-status statements as a current completion report.

See COD completion, security, schema and local acceptance for the current business implementation. No production deployment or data changes are included.

PARINAYA SILKS

React 19 + TypeScript + Vite 8 + Tailwind CSS 4 application. Public browsing uses clean URLs with legacy hash support. Administrator routes use real Firebase Authentication and server-managed permissions. Categories use Firestore; product management now uses the trusted catalog API and Cloudinary media flow. Customer email/password login, registration and password recovery use the shared Firebase Auth instance. COD checkout, customer orders and administrator business modules are implemented locally; see the completion guide for validation and release boundaries.

See Product CRUD configuration and acceptance for the current milestone and category management for its foundation. Product setup requires the local API server and server-only credentials. Owner rules/index publication, server configuration and application release remain necessary. No live test records or uploads were created.

Customer authentication

See customer authentication, private profile rules and release acceptance. The approved responsive login design is implemented with real Firebase flows and emulator-only automated accounts. Customer email verification is not required or sent; password-reset emails remain enabled. New registration now requires normalized mobile contact; the corresponding profile rule update must be coordinated with release. Owner review/publication of the narrow profile rules, authorized-domain/email-action settings and Vercel acceptance remain necessary. Release requirements and acceptance steps are documented in the linked guides.

See login appearance and mobile contact release guide. Owners can prepare separate desktop/mobile photos under Admin > Store Settings > Login Page Appearance and publish them with Save; the approved image remains the fallback.

Administrator access

See administrator setup and release instructions. Production /admin remains separate from public browsing and always requires real authorization. Both a server-assigned admin claim and an enabled adminAccess record are required. No customer admin shortcut or demo admin bypass remains.

Deployed restrictive rules were inspected and matched the repository baseline before this update. The mobile-field rule diff must be coordinated with the application release; local edits alone do not change deployed permissions.

Run npm.cmd run test:admin for isolated emulator tests. Provisioning is a separate trusted owner action: npm.cmd run admin:provision is read-only until explicit UID/email confirmation and --apply. The approved existing administrator was provisioned separately through the trusted procedure; provisioning is not part of storefront builds.

Firebase connection - Step 1

Confirmed target: project `parinaya-silks`, Cloud Firestore database `(default)`. Firebase setup and verification lists the exact local/Vercel variables and development-only read checks. The supplied browser configuration is in ignored `.env.local`; `.env.example` contains placeholders. Public category listings read enabled Firestore categories. Signed-in administrator access checks its own permission record. No mock data is migrated or seeded.

```
npm.cmd run firebase:check
npm.cmd run firebase:verify-build
```

App/client initialization is verified locally. The owner has supplied the existing permissive rules; the restrictive replacement and administrator access mechanism are documented in ADMIN_ACCESS.md. A signed-out document probe remains disabled. With the optional check path empty, `firebase:check` deliberately returns `check-path-missing` and a nonzero exit code. This is not a successful database read.

Local development

Node 22.22.2 and npm 10.9.7 were used for validation on Windows.

```
npm.cmd ci
npm.cmd run dev
npm.cmd run lint
npm.cmd test
npm.cmd run build
npm.cmd run preview
```

`lint` currently means TypeScript checking, not ESLint. Use test data only. Customer and administrator authentication use real Firebase. Automated customer account creation is restricted to the emulator suite. COD orders use the trusted API; online payments remain on hold.

The **storefront stays visible with clean empty states**. No sample catalogue, customer identity, order history, sale campaigns or fictional reporting is displayed. See content cleanup and review instructions.

There is no visibility flag or public simulator mode. `demoEnabled` is false in every mode. COD checkout and customer authentication use the trusted Firebase/API flows. Automated tests import fixtures directly from `tests/fixtures/commerce.ts`; the normal application never imports that dataset.

A one-time versioned migration removes only the 12 exact retired product IDs from the named legacy bag/wishlist keys. It preserves unknown rows, order/profile data, preferences and Firebase authentication storage. Those legacy records are not read into the normal website. See the cleanup guide for the exact keys and failure behavior.

Browser checks

With development running on `127.0.0.1:3000` and a default production preview on `127.0.0.1:4173`:

```
python scripts/browser-audit.py
python scripts/storefront-browser.test.py
python scripts/firebase-browser.test.py
```

The historical `browser-flows.py` delegates to the current empty-content checks. These scripts use the host's Python Playwright package and Chromium installation. They are not installed by npm. Route/viewport results and screenshots go to `.cache/browser`. Browser tests use isolated synthetic fixture records, not real customer information.

Project layout

- `src/App.tsx`: hash routing, shared shell and reactive state.
- `src/components`: storefront header, footer and mobile navigation.
- `src/views`: storefront screens and customer authentication route wrappers.
- `src/customer`: shared responsive authentication, session, profile and account flows.
- `src/admin`: admin screens and separate layout.
- `src/services/storeService.ts`: explicit empty/unconnected business repositories; no automatic seeding.
- `src/services/legacyDemoCleanup.ts`: narrow versioned legacy bag/wishlist cleanup.
- `src/types/index.ts`: shared data model.
- `src/utils`: shared totals, route normalization, fulfilment validation and tests.
- `src/config/runtime.ts`: public simulations remain disabled in every mode.
- `tests/fixtures/commerce.ts`: isolated fictional records for automated tests only.

See the takeover audit and Phase 3 plan for the historical baseline, screen coverage, known gaps and proposed future ownership model. Step 1 connection changes are described in Firebase setup. No live services, deployed rules or Vercel settings were changed.